

제출물: 과제 2 — AI 활용 검증 (무엇을 AI에 맡겼고, 어떻게 통제·증빙했는가) 시스템: Research-to-Backtest (상세 구현 정리: docs/SOLUTION_OVERVIEW.md) 증빙: submission/evidence/ai_usage_log.jsonl (실호출 기록) · run_state.json (승인 전이 이력) · 프롬프트 원문 4종 (src/research_backtest//prompts/_v1.txt, 버전 관리)

과제 2 — AI 활용 검증

1. 원칙: AI는 후보와 초안, 판단은 사람

이 시스템에서 AI(LLM)의 역할은 넷으로 한정되며, 그 외 모든 것 — 수치 계산, 근거 생성, 분석 관점, 투자 가설, 전략 승인, 결과 해석 — 은 Python 또는 사용자의 몫이다.

#	단계	AI가 하는 일	프롬프트 파일	받는 입력	사후 검증(코드)
1	분석 후보 정리	재무 Evidence를 사실·해석 후보·관계 가설·상충 근거로 분류 (CandidateAnalysis)	research/prompts/candidate_analysis_v1.txt	Python이 계산·검증한 Evidence 상위 60건	pydantic 스키마 + evidence_id 실존 검증(위반 시 재시도)
2	가설 후보 제시	참고용 가설 후보 2~4건 (HypothesisCandidate — 승인 가설 아님)	research/prompts/hypothesis_candidate_v1.txt	위 분석 후보 + Evidence	스키마 + evidence_id 검증 + generated_by / prompt_version 은 코드가 주입
3	전략 초안 변환	사용자가 승인한 가설 → 전략 DSL JSON 초안	quant/prompts/strategy_translation_v1.txt	승인 가설 (미승인은 게이트가 차단)	DSL 파서+컴파일러 통과, 종목·체결 규칙 (close→next_open) 강제 — 실패 사유를 피드백해 재시도
4	결과 설명 초안	백테스트 결과의 사실 서술 초안(유·불리 양면, 의견 금지)	quant/prompts/result_explanation_v1.txt	성과지표·강건성 요약	보고서에 [AI 초안] 태그로 격리 — 최종 해석은 사용자 섹션(\$11~14)과 물리적으로 분리

프롬프트에는 요구사항 원문(1804 §4)의 제약 7개 — 최종 의견 금지, evidence package 밖 사실 금지, 모든 finding에 evidence_id 연결, 사실/해석/관계 구분, 상충 근거 제시, 추정 금지, 복수 후보 제시 — 가 문면 그대로 들어 있고 전부 버전 파일로 관리된다.

2. 통제 장치 — "말로 하는 약속"이 아니라 코드

- 1. 승인 게이트: 파이프라인은 12-상태 머신으로 강제되며, AnalystView 검증 없이 가설 단계 진입 불가, 미승인 가설은 전략 변환 불가, 승인 기록 (StrategyReview) 없는 전략은 백테스트 진입점 첫 줄에서 거부(ApprovalGateError , CLI 종료 코드 4). 증빙: evidence/run_state.json 의 전이 이력(누가-언제-auto_approved 여부).
- 2. 저작 분리: AI 후보(hypothesis_candidates.json)와 사용자 가설 (human_investment_hypothesis.json)은 다른 pydantic 모델-다른 파일 — 합치는 것 자체가 불가능한 구조. 서술 콘텐츠에는 content_origin (7종) 저장, 보고서에는 섹션별 저작 태그([사용자 작성]/[Python 계산]/[AI 후보-초안 — 사용자 승인]) 표기.
- 3. 근거 봉쇄: LLM이 인용하는 모든 evidence_id는 Python이 만든 Evidence Store에 실존해야 하며, 지어낸 id는 검증 루프가 위반 목록을 피드백해 재 시도시킨다. Evidence 자체는 Point-in-Time 필터(공시 접수 다음 거래일 이후만)를 통과한 검증 수치만으로 구성된다 — API↔XBRL 교차검증(연간 100% MATCH) 위에서.
- 4. 신뢰 필드 주입: 모델명·프롬프트 버전 같은 기록 필드는 LLM 출력을 믿지 않고 코드가 주입한다. LLM은 도구 사용 금지(allowed_tools=[])-1턴으로 고정.

3. 실패 증빙 (run 20260715_152048_SK_HYNIX_INC)

모든 호출은 ai_usage_log.jsonl 에 남는다 — 이 run의 실행 기록 4건:

stage	model	prompt	입력 산출물	출력 산출물
candidate_analysis	claude-haiku-4-5-20251001	candidate_analysis v1	evidence_package.json	candidate_analysis.json
hypothesis_candidate	claude-haiku-4-5-20251001	hypothesis_candidate v1	evidence + 분석 후보	hypothesis_candidates.json
strategy_translation	claude-haiku-4-5-20251001	strategy_translation v1	human_investment_hypothesis.json	strategy_draft.json
result_explanation	claude-haiku-4-5-20251001	result_explanation v1	성과-강건성 요약	research_report.md §10.2

이 체인의 최종 산출물이 과제 1 보고서다: AI가 정리한 후보(§4)와 초안(§9-§10.2)이 어디까지이고, 사용자의 관점·가설·해석(§2·3·5~8·11~14)이 어디부터인지 보고서 안에서 태그로 구분된다. 모델은 저가형(Haiku)을 전체 ID로 판매 재현 가능하다 (configs/llm.yaml). 인증은 구독 OAuth 토큰이며 API 키와 동시 설정은 코드가 차단한다(의도치 않은 과금 방지).

4. 개발 과정의 AI 활용 (부록)

시스템 개발 자체도 AI 협업으로 진행했다: 설계·명세·병합·품질 게이트를 담당하는 메인 세션과, 마일스톤별 파일 소유권·인터페이스 계약이 명세 (docs/specs/)로 고정된 워크트리 격리 병렬 구현 에이전트의 분업이다(결정 기록 D8). 4개 웨이브·12개 병렬 트랙, 병합 충돌 1건(import), 최종 품질 게이트 pytest 837 passed(실 API·실데이터·live LLM 포함)·mypy strict 179파일 0 이슈. 에이전트의 명세 이탈은 전부 사유와 함께 기록·심사했으며, 그 과정 자체가 docs/PROGRESS.md #1~#5에 스냅샷으로 남아 있다.

5. 재현 방법

```
make install && cp .env.example .env # DART·KRX·CLAUDE_CODE_OAUTH_TOKEN 입력
r2b collect-financials --company 000660 --from-year 2015 --to-year 2025 --include-xbrl
r2b collect-market --company 000660 && r2b build-financials --company 000660
r2b research --company 000660 --as-of-date 2025-12-31 # AI 후보 (호출 기록 시작)
# 사용자 단계: create-analyst-view → create-hypothesis → (AI 초안) → approve-strategy
r2b backtest --run-id <run_id> && r2b submit-interpretation --run-id <run_id> --input ...
r2b generate-report --run-id <run_id> # 15-섹션 보고서
```

Streamlit UI(python -m streamlit run src/research_backtest/app/streamlit_app.py)도 동일한 게이트·검증 코드로 같은 흐름을 제공한다.